

Lokaler Build-Befehl

1. Scraper ausführen (erstellt docs_site/data.js):

```
`bash
```

```
python scrape_docs.py
```

```
`
```

2. PDF Bericht generieren (erstellt docs_site/documentation_report.pdf):

```
`bash
```

```
python generate_pdf.py
```

```
`
```

CI/CD Pipeline (GitHub Actions)

Bei jedem Push auf den main-Branch baut die Pipeline .github/workflows/docs.yml die Webseite und das PDF automatisch. Wenn Sie einen Commit pushen, der bereits kompilierte Änderungen enthält, nutzen Sie [skip ci] im Commit-Betreff, um endlose Build-Loops zu verhindern.

Datei: [doc/Entscheidungen_Echtzeit_Training.md](#)

Architekturentscheidungen: Echtzeit-Training & Bewegungserfassung

Dieses Dokument dokumentiert die Architekturentscheidungen zur Implementierung des interaktiven Trainings-Workflows (Bezug: [UC-2](#), Schritte 3 & 4) in der MoveLink Mobile App.

Übersicht der Phasen und Entscheidungen

flowchart TD

```
Start[User klickt 'Start'] --> P1[Phase 1: Demo & Countdown\nLottie Animation]
P1 --> P2[Phase 2: Live-Erfassung\nComplementary Filter & SVG UI]
P2 --> P3[Phase 3: Trajektorien-Vergleich\nZustandsmaschine / FSM]
P3 --> P4[Phase 4: Multi-Sensory Feedback\nexpo-haptics & Sound]
```

Phase 1: Grafische Übungsdemonstration

- In Arduino / C++ Dateien (embedded/):*

```
/*
 * @implements FA5, NF1
 * Liest Sensorwerte mit 50Hz aus und wendet einen Tiefpassfilter an.
 */
void loop() {
    // Sensorsignal...
}
```

3. C4-Architektur-Modellierung (Metadaten-Blöcke)

Jedes Architektur-Dokument in Markdown muss Informationen über die zugehörige C4-Ebene und die Deployability enthalten.

Metadaten-Header in Markdown

Fügen Sie ganz oben in der entsprechenden Architektur-Markdown-Datei (z. B. app/architecture.md) einen HTML-Kommentarblock mit folgenden Keys hinzu:

```
<!--
C4-Ebene: Container
Deployable: Ja
-->
```

- Erlaubte Werte:*
- C4-Ebene: System-Context, Container, Component
- Deployable: Ja / Nein (oder Yes / No)

Registrierung im C4 Model Explorer

Wenn ein neuer Container oder eine neue Komponente hinzugefügt wird, muss diese auch in der C4_DATA-Struktur am Ende von docs_site/app.js registriert werden:

1. Tragen Sie das Element unter containers.elements oder components.[container_id].elements ein.
 2. Definieren Sie dessen Verbindungen (Connectoren) im zugehörigen connections-Array.
 3. Ergänzen Sie die Dateizuordnung in der Funktion getC4ElementForFile in docs_site/app.js, damit die E2E-Flussdiagramme die Datei dem neuen C4-Element zuweisen.
-

4. Build-Prozess & Pipeline

Nach jeder Änderung an der Dokumentation oder den @implements-Kommentaren im Quellcode müssen die Kompilierungsskripte ausgeführt werden:

- In Arduino / C++ Dateien (embedded/):*

```
/*
 * @implements FA5, NF1
 * Liest Sensorwerte mit 50Hz aus und wendet einen Tiefpassfilter an.
 */
void loop() {
    // Sensorsignal...
}
```

3. C4-Architektur-Modellierung (Metadaten-Blöcke)

Jedes Architektur-Dokument in Markdown muss Informationen über die zugehörige C4-Ebene und die Deployability enthalten.

Metadaten-Header in Markdown

Fügen Sie ganz oben in der entsprechenden Architektur-Markdown-Datei (z. B. app/architecture.md) einen HTML-Kommentarblock mit folgenden Keys hinzu:

```
<!--
C4-Ebene: Container
Deployable: Ja
-->
```

- Erlaubte Werte:*
- C4-Ebene: System-Context, Container, Component
- Deployable: Ja / Nein (oder Yes / No)

Registrierung im C4 Model Explorer

Wenn ein neuer Container oder eine neue Komponente hinzugefügt wird, muss diese auch in der C4_DATA-Struktur am Ende von docs_site/app.js registriert werden:

1. Tragen Sie das Element unter containers.elements oder components.[container_id].elements ein.
 2. Definieren Sie dessen Verbindungen (Connectoren) im zugehörigen connections-Array.
 3. Ergänzen Sie die Dateizuordnung in der Funktion getC4ElementForFile in docs_site/app.js, damit die E2E-Flussdiagramme die Datei dem neuen C4-Element zuweisen.
-

4. Build-Prozess & Pipeline

Nach jeder Änderung an der Dokumentation oder den @implements-Kommentaren im Quellcode müssen die Kompilierungsskripte ausgeführt werden:


```
    }
    break;
  case 'peak':
    if (absoluteAngle < targetAngle - 5) {
      setState('returning');
    }
    break;
  case 'returning':
    if (absoluteAngle < 15) {
      setState('start');
      setReps((r) => r + 1); // Zähler erhöhen
    }
    break;
  }
};

const playSuccessSound = async () => {
  const { sound } = await Audio.Sound.createAsync(
    require('@assets/sounds/success.mp3')
  );
  await sound.playAsync();
};

return { angle, reps, state, updateMeasurement };
}
```

Datei: [embedded/architecture.md](#)

MoveLink Embedded Firmware - Container-Architektur

Dieses Dokument beschreibt die Embedded Sensor-Firmware als eigenständige, deploybare Einheit im C4-Modell.

C4-Architektur-Ebene

- C4-Ebene: Container
- Deployable: Ja
- Deployment-Artefakt: Binär-Firmware (flashed via USB/Serial)
- Technologie-Stack: Arduino C/C++, LSM6DS3 IMU Library, Edge Impulse SDK, Bluetooth Low Energy

Beschreibung

Die Sensor-Firmware läuft auf dem XIAO nRF52840 Sense Controller. Sie erfasst Beschleunigungs- und Rotationsdaten über den integrierten LSM6DS3-Sensor mit einer festen Abtastrate (50Hz), wendet Signalfilterungen zur Rauschunterdrückung an und streamt die Datenpakete als binäres Array via BLE

Characteristics an die Mobile App. Alternativ führt sie Edge-Impulse-Inferenzmodelle direkt auf dem Mikrocontroller aus, um Trainingsübungen (z.B. Bizeps-Curls) lokal zu klassifizieren und Fehler über die integrierten RGB-LEDs anzuzeigen.

Komponenten in diesem Container

Die Sensor-Firmware besteht aus folgenden logischen Komponenten:

1. [Sensordatenerfassung (Loop)](file:///c:/Users/erlin/repo/movelink/embedded/components/sensordatenerfassung/architecture.md): Liest kontinuierlich Beschleunigung (X, Y, Z) und Gyroskop (X, Y, Z). (Erfüllt: [FA5](#), [NF1](#))
2. [Inferenz-Engine (Edge Impulse)](file:///c:/Users/erlin/repo/movelink/embedded/components/inferenz_engine/architecture.md): Klassifiziert Übungsausführungen lokal auf dem Chip. (Erfüllt: [FA5](#), [FA9](#))
3. [LED- & Display-Controller](file:///c:/Users/erlin/repo/movelink/embedded/components/led_display_controller/architecture.md): Bietet direktes visuelles Feedback an den Nutzer bei Fehlern. (Erfüllt: [FA9](#))
4. [BLE-Streamer](file:///c:/Users/erlin/repo/movelink/embedded/components/ble_streamer/architecture.md): Überträgt die erfassten Daten an den App-Container. (Erfüllt: [FA3](#), [FA5](#))
5. [Gehäuse](file:///c:/Users/erlin/repo/movelink/embedded/components/gehaeuse/architecture.md): Bietet physischen Schutz, sodass das Tragen erleichtert wird. (Erfüllt: [R2](#))

Abwägungen

- Lokale Auswertung vs. Cloud-Streaming: Das Ausführen der Inferenz-Engine direkt auf dem Xiao-Controller minimiert die Latenz ([NF1](#)) und spart Bandbreite bei der Funkübertragung.
- Energiebedarf: OLED-Display und kontinuierliche Sensordatenerfassung verbrauchen signifikant Energie, weshalb Akkulaufzeiten durch Cooldown-Zeiten und Schlafmodi im Idle optimiert werden müssen.

Datei: [embedded/components/ble_streamer/architecture.md](#)

<!--

C4-Ebene: Component

Deployable: Nein

• ->

BLE-Streamer

Diese Komponente überträgt die erfassten 6-Achsen-Sensordaten (Beschleunigung und Rotation) in Echtzeit per Bluetooth Low Energy (BLE) an die Mobile App.

Begründung

- **Haptik:** Während des Trainings schaut der Trainierende oft nicht direkt auf den Bildschirm (z. B. bei Kniebeugen). Ein kurzes haptisches Signal beim Erreichen des Peak-Winkels und beim Abschluss der Wiederholung ist daher essenziell für die Usability.
 - **Visuelle Belohnung:** Der Fortschrittsring wechselt beim Erreichen des Zielwinkels seine Farbe von Blau/Türkis zu einem leuchtenden Smaragdgrün, um ein sofortiges Erfolgsgefühl zu vermitteln.
-

Technische Blaupause (TypeScript-Referenz)

Folgender Code skizziert die Realisierung des Tracking-Hooks:

```
import { useState, useRef } from 'react';
import * as Haptics from 'expo-haptics';
import { Audio } from 'expo-av';
import { IMUReading } from '@store';

export type ExerciseState = 'start' | 'moving' | 'peak' | 'returning';

export function useExerciseTracker(targetAngle = 90) {
  const [angle, setAngle] = useState(0);
  const [reps, setReps] = useState(0);
  const [state, setState] = useState<ExerciseState>('start');

  const lastTime = useRef<number>(Date.now());
  const currentAngle = useRef<number>(0);

  const updateMeasurement = async (reading: IMUReading) => {
    const now = Date.now();
    const dt = (now - lastTime.current) / 1000.0;
    lastTime.current = now;

    // 1. Beschleunigungswinkel berechnen (z.B. Pitch)
    const accelAngle = Math.atan2(reading.accelY, reading.accelZ) * (180 / Math.PI);

    // 2. Gyroskop-Integration & Komplementärfilter
    const alpha = 0.96;
    currentAngle.current = alpha * (currentAngle.current + reading.gyroX * dt) + (1 - alpha) * accelAngle;

    const absoluteAngle = Math.abs(currentAngle.current);
    setAngle(absoluteAngle);

    // 3. FSM für Wiederholungs- & Peak-Erkennung
    switch (state) {
      case 'start':
        if (absoluteAngle > 20) {
          setState('moving');
        }
        break;
      case 'moving':
        if (absoluteAngle >= targetAngle) {
          setState('peak');
          // Sofortiges haptisches Feedback bei Erfolg
          await Haptics.notificationAsync(Haptics.NotificationFeedbackType.Success);
          // Optionaler Sound
          playSuccessSound();
        }

```



```
    }
    break;
  case 'peak':
    if (absoluteAngle < targetAngle - 5) {
      setState('returning');
    }
    break;
  case 'returning':
    if (absoluteAngle < 15) {
      setState('start');
      setReps((r) => r + 1); // Zähler erhöhen
    }
    break;
  }
};

const playSuccessSound = async () => {
  const { sound } = await Audio.Sound.createAsync(
    require('@assets/sounds/success.mp3')
  );
  await sound.playAsync();
};

return { angle, reps, state, updateMeasurement };
}
```

Datei: [embedded/architecture.md](#)

MoveLink Embedded Firmware - Container-Architektur

Dieses Dokument beschreibt die Embedded Sensor-Firmware als eigenständige, deploybare Einheit im C4-Modell.

C4-Architektur-Ebene

- C4-Ebene: Container
- Deployable: Ja
- Deployment-Artefakt: Binär-Firmware (flashed via USB/Serial)
- Technologie-Stack: Arduino C/C++, LSM6DS3 IMU Library, Edge Impulse SDK, Bluetooth Low Energy

Beschreibung

Die Sensor-Firmware läuft auf dem XIAO nRF52840 Sense Controller. Sie erfasst Beschleunigungs- und Rotationsdaten über den integrierten LSM6DS3-Sensor mit einer festen Abtastrate (50Hz), wendet Signalfilterungen zur Rauschunterdrückung an und streamt die Datenpakete als binäres Array via BLE

Characteristics an die Mobile App. Alternativ führt sie Edge-Impulse-Inferenzmodelle direkt auf dem Mikrocontroller aus, um Trainingsübungen (z.B. Bizeps-Curls) lokal zu klassifizieren und Fehler über die integrierten RGB-LEDs anzuzeigen.

Komponenten in diesem Container

Die Sensor-Firmware besteht aus folgenden logischen Komponenten:

1. [Sensordatenerfassung (Loop)](file:///c:/Users/erlin/repo/movelink/embedded/components/sensordatenerfassung/architecture.md): Liest kontinuierlich Beschleunigung (X, Y, Z) und Gyroskop (X, Y, Z). (Erfüllt: [FA5](#), [NF1](#))
2. [Inferenz-Engine (Edge Impulse)](file:///c:/Users/erlin/repo/movelink/embedded/components/inferenz_engine/architecture.md): Klassifiziert Übungsausführungen lokal auf dem Chip. (Erfüllt: [FA5](#), [FA9](#))
3. [LED- & Display-Controller](file:///c:/Users/erlin/repo/movelink/embedded/components/led_display_controller/architecture.md): Bietet direktes visuelles Feedback an den Nutzer bei Fehlern. (Erfüllt: [FA9](#))
4. [BLE-Streamer](file:///c:/Users/erlin/repo/movelink/embedded/components/ble_streamer/architecture.md): Überträgt die erfassten Daten an den App-Container. (Erfüllt: [FA3](#), [FA5](#))
5. [Gehäuse](file:///c:/Users/erlin/repo/movelink/embedded/components/gehaeuse/architecture.md): Bietet physischen Schutz, sodass das Tragen erleichtert wird. (Erfüllt: [R2](#))

Abwägungen

- Lokale Auswertung vs. Cloud-Streaming: Das Ausführen der Inferenz-Engine direkt auf dem Xiao-Controller minimiert die Latenz ([NF1](#)) und spart Bandbreite bei der Funkübertragung.
- Energiebedarf: OLED-Display und kontinuierliche Sensordatenerfassung verbrauchen signifikant Energie, weshalb Akkulaufzeiten durch Cooldown-Zeiten und Schlafmodi im Idle optimiert werden müssen.

Datei: [embedded/components/ble_streamer/architecture.md](#)

<!--

C4-Ebene: Component

Deployable: Nein

• ->

BLE-Streamer

Diese Komponente überträgt die erfassten 6-Achsen-Sensordaten (Beschleunigung und Rotation) in Echtzeit per Bluetooth Low Energy (BLE) an die Mobile App.

C4-Architektur-Ebene

- C4-Ebene: Component
- Deployable: Nein (Läuft als Teil des Sensor Firmware Containers)

Beschreibung

Der BLE-Streamer initialisiert den nRF52840-Bluetooth-Stack, stellt einen GATT-Service mit einer IMU-Characteristic bereit und sendet die gefilterten Sensor-Messwerte (Beschleunigung in X, Y, Z und Drehraten in X, Y, Z) als binären 24-Byte-Puffer (6 float-Werte) an verbundene Clients.

GATT Profile & UUIDs

- Service-UUID: 12345678-1234-1234-1234-123456789012
- Characteristic-UUID: 12345678-1234-1234-1234-123456789013 (BLERead | BLENotify)
- Paketformat: 24 Bytes (6 float-Werte, IEEE 754 float32, little-endian):

[accelX, accelY, accelZ, gyroX, gyroY, gyroZ]

Implementierung & Traceability

- Implementiert in: [BLEStreamer.cpp](file:///c:/Users/erlin/repo/movelink/embedded/components/ble_streamer/BLEStreamer.cpp)
- Erfüllt Anforderungen:
- **FA3**: Verbindungsaufbau: Ermöglicht der Mobile App den Bluetooth-Aufbau und das Pairing.
- **FA5**: Datenstrom-Verarbeitung: Streamt die Rohdaten kontinuierlich via BLE Characteristics.

Datenfluss

flowchart LR

```
IMU[Sensor-Rohwerte] -->|ax, ay, az, gx, gy, gz| Streamer[BLE-Streamer Component]
Streamer -->|GATT Notification (24 Bytes)| App[Mobile App useBLE Hook]
```

Datei: embedded/components/gehause/architecture.md

<!--

C4-Ebene: Component

Deployable: Nein

- ->

Gehäuse (Enclosure)

Diese Komponente beschreibt das physische, schützende 3D-Druck-Gehäuse des Sensors.

C4-Architektur-Ebene

- C4-Ebene: Component
- Deployable: Nein (Physische Schutzhülle, keine Software-Ausführung)

Beschreibung

Das Gehäuse umschließt den XIAO-Mikrocontroller sowie die Peripherie (Display, Batterie). Es bietet Befestigungslaschen für ein standardmäßiges 20-mm-Sportarmband, um den Sensor stabil am Arm des Nutzers zu fixieren.

Technische & Physische Parameter

- Gesamtmaße: 48 mm (Länge) x 24 mm (Breite) x 16 mm (Höhe)
- Außenwandstärke: 2.0 mm
- Schließmechanismus: Schnapp-Deckel (Lippe & Snap Bumps mit 0.2 mm Toleranz)
- Komfort: Abgerundete Ecken (Bevel-Breite 1.5 mm), um Druckstellen beim Tragen zu vermeiden.
- Aussparungen: Integrierter USB-C-Port zur Programmierung und Akku-Ladung.

Implementierung & Traceability

- Implementiert in: [Gehäuse.py](file:///c:/Users/erlin/repo/movelink/embedded/src/Gehäuse.py) (Blender Python API)
- Erfüllt Anforderungen:
- **R2**: Physisches Gehäuse: Stabile Fixierung des Sensors am Arm, Schutz gegen Schweiß und Erschütterungen.

Schnittstellen

Das Gehäuse hat keine softwareseitigen Verbindungen, interagiert aber mechanisch mit:

- Mikrocontroller (XIAO nRF52840 Sense): Durch Passform und Aussparungen fixiert.

- Armband (20mm): Wird durch die integrierten Lug-Slots gefädelt.

Datei: [embedded/components/inferenz_engine/architecture.md](#)

<!--

C4-Ebene: Component

Deployable: Nein

• ->

Inferenz-Engine (Edge Impulse)

Diese Komponente klassifiziert Übungsausführungen in Echtzeit direkt auf dem Mikrocontroller (Edge Computing).

C4-Architektur-Ebene

- C4-Ebene: Component
- Deployable: Nein (Läuft als Teil des Sensor Firmware Containers)

Beschreibung

Die Inferenz-Engine führt ein CNN-Klassifikationsmodell aus, das über Edge Impulse trainiert und als Arduino-Bibliothek in die Firmware integriert wurde. Es analysiert die 6-Achsen-Bewegungsdaten auf spezifische Übungsqualitäten und Fehlerbilder.

Technische Details

- Modelltyp: Convolutional Neural Network (CNN)
- Erkannte Klassen:
 - idle: Keine Übungsausführung / Ruhezustand.
 - curl_sauber: Korrekt ausgeführter Bizeps-Curl.
 - fehler_rotation: Fehlerhafte Ausführung durch Rotation des Handgelenks.
 - fehler_ellbogen: Fehlerhafte Ausführung durch Bewegung des Ellbogens.
- Anomalieerkennung: Optionaler K-Means-Clustering-Block zur Erkennung unbekannter Bewegungen.

Implementierung & Traceability

- Implementiert in: [Executable.ino](file:///c:/Users/erlin/repo/movelink/embedded/src/Executable.ino) (unter Einbindung von Erlind-project-1_inferencing.h)
- Erfüllt Anforderungen:
- **FA5:** Datenstrom-Verarbeitung: Analyse des kontinuierlichen Datenstroms.
- **FA9:** Biofeedback und Auswertung: Liefert die Grundlage für das unmittelbare Feedback (Erkennung sauberer vs. fehlerhafter Curls).

Datenfluss

flowchart TD

```
DSP[DSP Puffer (6 Achsen)] -->|run_classifier| Model[CNN Inferenzmodell]
Model -->|Wahrscheinlichkeiten| Eval[Klassenauswertung]
Eval -->|Bester Treffer + Score| Output[Feedback & PC-JSON-Stream]
```

Datei: [embedded/components/led_display_controller/architecture.md](#)

<!--

C4-Ebene: Component

Deployable: Nein

• ->

LED- & Display-Controller

Diese Komponente gibt dem Trainierenden direktes visuelles Feedback zur Qualität der Übungsausführung.

C4-Architektur-Ebene

- C4-Ebene: Component
- Deployable: Nein (Läuft als Teil des Sensor Firmware Containers)

Beschreibung

Der Controller steuert das SSD1306 OLED-Display sowie die integrierten RGB-LEDs des XIAO-Boards basierend auf den Klassifikationsergebnissen der Inferenz-Engine.

Feedback-Logik

- Ruhemodus (idle oder Konfidenz < 60%):
- RGB-LED: Blau (Pin 12 auf LOW)
- Display: Zeigt Status: IDLE an.
- Saubere Ausführung (curl_sauber):
- RGB-LED: Grün (Pin 13 auf LOW)
- Display: Zeigt Curl: PERFEKT an.
- Fehlerhafte Ausführung (z. B. fehler_rotation, fehler_ellbogen):
- RGB-LED: Rot (Pin 11 auf LOW)
- Display: Zeigt Achtung: FEHLER an.

Implementierung & Traceability

- Implementiert in: [Executable.ino](file:///c:/Users/erlin/repo/movelink/embedded/src/Executable.ino) (unter Verwendung der U8x8-Bibliothek)
- Erfüllt Anforderungen:
- **FA9:** Biofeedback und Auswertung: Ermöglicht sofortiges visuelles Biofeedback direkt an der Sensor-Hardware.

Kontrollfluss

flowchart TD

```

Result[Inferenz-Ergebnis] --> Check{Welche Klasse?}
Check -->|idle / <60%| Idle[Blau leuchten / 'IDLE' anzeigen]
Check -->|curl_sauber| Perfect[Grün leuchten / 'PERFEKT' anzeigen]
Check -->|Fehlerklasse| Error[Rot leuchten / 'FEHLER' anzeigen]
    
```

Datei: [embedded/components/sensordatenerfassung/architecture.md](#)

<!--

C4-Ebene: Component

Deployable: Nein

- ->

Sensordatenerfassung (Loop)

Diese Komponente liest kontinuierlich die Rohwerte des Beschleunigungssensors und Gyroskops aus.

C4-Architektur-Ebene

- C4-Ebene: Component
- Deployable: Nein (Läuft als Teil des Sensor Firmware Containers)

Beschreibung

Die Sensordatenerfassung liest in einer festen Schleife (Loop) die 6-Achsen-IMU-Werte (Beschleunigung und Drehung) des LSM6DS3-Sensors über den I2C-Bus ein.

Technische Details

- Abtastrate: 50 Hz (gesteuert durch präzises Timing in Microsekunden)
- Signal-Clamping: Beschleunigungswerte werden auf max. ± 2.0 G gedämpft/geclamt.
- Konvertierung: Die Werte werden in die SI-Einheit m/s^2 konvertiert ($1\text{G} = 9.80665\text{m/s}^2$).
- Verwendete Hardware: LSM6DS3 IMU auf dem XIAO nRF52840 Sense.

Implementierung & Traceability

- Implementiert in: [Executable.ino](file:///c:/Users/erlin/repo/movelink/embedded/src/Executable.ino)
- Erfüllt Anforderungen:
 - **FA5**: Datenstrom-Verarbeitung: Die Sensordaten werden kontinuierlich erfasst und für die Klassifikation aufbereitet.
 - **NF1**: Latenz: Durch die hardwarenahe I2C-Abfrage und das Vermeiden von blockierenden Delays wird eine niedrige E2E-Latenz ermöglicht.

Datenfluss

flowchart LR

```

IMU[LSM6DS3 Sensor] -->|I2C Rohdaten| Loop[Loop in Executable.ino]
Loop -->|1. Clamping auf 2G| Calc[Skalierung & m/s^2 Konvertierung]
Calc -->|2. Puffer befüllen| DSP[Edge Impulse DSP Puffer]
    
```


Datei: [embedded/src/architecture.md](#)

Embedded Sensor-Firmware

Die Firmware liest Sensorwerte aus und überträgt diese über Bluetooth Low Energy (BLE) an die App.

Datenfluss Firmware

flowchart TD

```

    Sensor[MPU6050 Beschleunigungssensor] -->|I2C Rohdaten| Arduino[Arduino / ESP32 Controller]
    Arduino -->|Signalfilterung & Skalierung| BLE[Bluetooth Low Energy Characteristic]
    BLE -->|Notifikationen / Byte-Array| App[Mobile App]
  
```

- Sensordatenerfassung: Erfolgt in einer festen Frequenz (z.B. 50Hz).
- Filterung: Tiefpassfilter zur Rauschminderung auf dem Mikrocontroller.
- BLE-Transfer: Effiziente Übertragung als binäres Datenpaket.

2. Requirements vs. Use Cases Matrix

Die folgende Matrix veranschaulicht die Beziehungen zwischen den definierten funktionalen Anforderungen (FA) / nicht-funktionalen Anforderungen (NF) / Rahmenbedingungen (R) und den Use Cases (UC).

| Anforderung / ID | UC-1 | UC-2 | UC-3 |
|---|------|------|------|
| FA1: Dashboard und Navigation Das System muss eine... | X | X | X |
| FA2: Bluetooth LE Signalstärke** Das System muss d... | - | - | - |
| FA3: BLE Verbindungsaufbau (UC-1)** Das System mus... | X | - | - |
| FA4: Trainings-Detailansicht Das System muss eine ... | - | X | - |
| FA5: Datenstrom-Verarbeitung Das System muss konti... | - | X | - |
| FA6: Echtzeit-Visualisierung Das System muss die e... | - | X | - |
| FA7: Historische Analyse Das System muss historisc... | - | - | X |
| FA8: Übungs-Demonstration Das System muss eine gra... | - | X | - |
| FA9: Biofeedback und Auswertung Das System muss de... | - | X | - |
| NF1: Latenz Die End-to-End-Latenz von der physisch... | - | - | - |
| NF2: Zuverlässigkeit und Reconnect Bei Verbindungs... | - | - | - |
| NF3: Benutzbarkeit (Usability) Das Bluetooth-Pairi... | - | - | - |
| R1: Plattform-Kompatibilität Die mobile Applikati... | - | - | - |
| R2: Physisches Gehäuse Für das Trainingsgeräte br... | - | - | - |

Hinweis: Ein 'X' kennzeichnet eine direkte Verknüpfung im Pflichtenheft bzw. den Anforderungsdokumenten.

3. End-to-End Rückverfolgbarkeits-Baum

Hier ist der vollständige Trace-Pfad von den Anwendungsfällen über die funktionalen Anforderungen bis hin zu den konkreten Code-Implementierungen (z. B. React Native-Dateien, Arduino-Skripte) dargestellt.

Anwendungsfall UC-1: Trainingsgerät verbinden

- Anforderung FA1: Dashboard und Navigation Das System muss eine Navigationsstruktur (Reiter/Menü) für Hardwaregeräte, Trainings und vergangene Trainingseinheiten bereitstellen. *(Bezug: UC-1, UC-2, UC-3)*

- Implementiert in: app/app/(tabs)/_layout.tsx (Zeile 2)

Kontext: * @implements FA1

- Implementiert in: app/app/(tabs)/index.tsx (Zeile 2)

Kontext: * @implements FA1, FA4, FA6, FA8, FA9

- Implementiert in: doc/AI_DOCUMENTATION_GUIDE.md (Zeile 16)

Kontext: * **FA-X***: Funktionale Anforderungen (z. B. `FA1`)

- Implementiert in: app/architecture.md (Zeile 23)

Kontext: 1. **SideNav**: Navigationskomponente für die App-Steuerung. (Erfüllt: FA1)

- Anforderung FA3: BLE Verbindungsaufbau (UC-1)** Das System muss eine Bluetooth Low Energy Verbindung zum Sensor herstellen. ``

- Implementiert in: embedded/components/ble_streamer/BLEStreamer.h (Zeile 6)

Kontext: // @implements FA3, FA5

- Implementiert in: embedded/components/ble_streamer/BLEStreamer.cpp (Zeile 1)

Kontext: // @implements FA3, FA5

- Implementiert in: app/components/SensorCard.tsx (Zeile 2)

Kontext: * @implements FA2, FA3, NF3

- Implementiert in: app/hooks/useBLE.ts (Zeile 2)

Kontext: * @implements FA3, FA5, NF2

- Implementiert in: embedded/architecture.md (Zeile 19)

Kontext: 4. **[BLE-Streamer](file:///c:/Users/erlin/repo/movelink/embedded/components/ble_streamer/architecture.md)**: Überträgt die erfassten Daten an den App-Container. (Erfüllt: FA3, FA5)

- Implementiert in: embedded/components/ble_streamer/architecture.md (Zeile 26)

Kontext: * **FA3: Verbindungsaufbau***: Ermöglicht der Mobile App den Bluetooth-Aufbau und das Pairing.

- Implementiert in: doc/Requirements.md (Zeile 15)

Kontext: **FA3**: Verbindungsaufbau

- Implementiert in: doc/AI_DOCUMENTATION_GUIDE.md (Zeile 54)

Kontext: // @implements FA2, FA3, NF2

- Implementiert in: app/architecture.md (Zeile 24)

Kontext: 2. **SensorCard**: Verwaltung der BLE-Geräteverbindung und Pairing. (Erfüllt: FA2, FA3, NF3)

- Implementiert in: app/architecture.md (Zeile 27)

Kontext: 5. **BLE-Hook (useBLE)**: Kapselt die Bluetooth-Gerätekommunikation und den Reconnect.

(Erfüllt: FA3, FA5, NF2)

Anwendungsfall UC-2: Echtzeit-Training überwachen

- Anforderung FA1: Dashboard und Navigation Das System muss eine Navigationsstruktur (Reiter/Menü) für Hardwaregeräte, Trainings und vergangene Trainingseinheiten bereitstellen. *(Bezug: UC-1, UC-2, UC-3)*

- Implementiert in: app/app/(tabs)/_layout.tsx (Zeile 2)

Kontext: * @implements FA1

- Implementiert in: app/app/(tabs)/index.tsx (Zeile 2)

Kontext: * @implements FA1, FA4, FA6, FA8, FA9

- Implementiert in: doc/AI_DOCUMENTATION_GUIDE.md (Zeile 16)

Kontext: * **FA-X**: Funktionale Anforderungen (z. B. `FA1`)

- Implementiert in: app/architecture.md (Zeile 23)

Kontext: 1. **SideNav**: Navigationskomponente für die App-Steuerung. (Erfüllt: FA1)

- Anforderung FA4: Trainings-Detailansicht Das System muss eine detaillierte Ansicht für ein ausgewähltes Training anzeigen. *(Bezug: UC-2)*

- Implementiert in: app/app/(tabs)/index.tsx (Zeile 2)

Kontext: * @implements FA1, FA4, FA6, FA8, FA9

- Anforderung FA5: Datenstrom-Verarbeitung Das System muss kontinuierliche Bewegungsdatenströme vom Sensor empfangen, filtern und verarbeiten können. *(Bezug: UC-2)*

- Implementiert in: embedded/src/Executable.ino (Zeile 2)

Kontext: * @implements FA5, FA9, NF1

- Implementiert in: embedded/components/ble_streamer/BLEStreamer.h (Zeile 6)

Kontext: // @implements FA3, FA5

- Implementiert in: embedded/components/ble_streamer/BLEStreamer.h (Zeile 9)

Kontext: // @implements FA5

- Implementiert in: embedded/components/ble_streamer/BLEStreamer.cpp (Zeile 1)

Kontext: // @implements FA3, FA5

- Implementiert in: embedded/components/sensordatenerfassung/IMUReader.h (Zeile 6)

Kontext: // @implements FA5, NF1

- Implementiert in: embedded/components/sensordatenerfassung/IMUReader.h (Zeile 9)

Kontext: // @implements FA5, NF1

- Implementiert in: embedded/components/sensordatenerfassung/IMUReader.cpp (Zeile 1)

Kontext: // @implements FA5, NF1

- Implementiert in: embedded/components/sensordatenerfassung/IMUReader.cpp (Zeile 15)

Kontext: // @implements FA5, NF1

- Implementiert in: embedded/components/sensordatenerfassung/IMUReader.cpp (Zeile 20)

Kontext: // @implements FA5, NF1

- Implementiert in: embedded/components/inferenz_engine/InferenceEngine.h (Zeile 6)

Kontext: // @implements FA5, FA9

- Implementiert in: embedded/components/inferenz_engine/InferenceEngine.cpp (Zeile 1)

Kontext: // @implements FA5, FA9

- Implementiert in: embedded/components/inferenz_engine/InferenceEngine.cpp (Zeile 7)

Kontext: // @implements FA5, FA9

- Implementiert in: app/store/index.ts (Zeile 2)

Kontext: * @implements FA5, FA9

- Implementiert in: app/hooks/useBLE.ts (Zeile 2)

Kontext: * @implements FA3, FA5, NF2

- Implementiert in: app/hooks/useWebSocket.ts (Zeile 2)

Kontext: * @implements FA5

- Implementiert in: embedded/architecture.md (Zeile 16)

Kontext: 1. **[Sensordatenerfassung (Loop)]**(file:///c:/Users/erlin/repo/movelink/embedded/component
s/sensordatenerfassung/architecture.md)******: Liest kontinuierlich Beschleunigung (X, Y, Z) und
Gyroskop (X, Y, Z). (Erfüllt: FA5, NF1)

- Implementiert in: embedded/architecture.md (Zeile 17)

Kontext: 2. **[Inferenz-Engine (Edge Impulse)]**(file:///c:/Users/erlin/repo/movelink/embedded/compon
ents/inferenz_engine/architecture.md)******: Klassifiziert Übungsausführungen lokal auf dem Chip.
(Erfüllt: FA5, FA9)

- Implementiert in: embedded/architecture.md (Zeile 19)

Kontext: 4. **[BLE-Streamer]**(file:///c:/Users/erlin/repo/movelink/embedded/components/ble_streamer/
architecture.md)******: Überträgt die erfassten Daten an den App-Container. (Erfüllt: FA3, FA5)

- Implementiert in: embedded/components/ble_streamer/architecture.md (Zeile 27)

Kontext: * **FA5: Datenstrom-Verarbeitung********: Streamt die Rohdaten kontinuierlich via BLE
Characteristics.

- Implementiert in: embedded/components/sensordatenerfassung/architecture.md (Zeile 26)

Kontext: - **FA5: Datenstrom-Verarbeitung********: Die Sensordaten werden kontinuierlich erfasst und für
die Klassifikation aufbereitet.

- Implementiert in: embedded/components/inferenz_engine/architecture.md (Zeile 29)

Kontext: - **FA5: Datenstrom-Verarbeitung********: Analyse des kontinuierlichen Datenstroms.

- Implementiert in: doc/AI_DOCUMENTATION_GUIDE.md (Zeile 63)

Kontext: * @implements FA5, NF1

- Implementiert in: doc/UseCases.md (Zeile 33)

Kontext: ****Ausgabe********: Die App visualisiert die Bewegung in Echtzeit, vergleicht sie mit der
Zielvorgabe und gibt positives Feedback bei korrekter Ausführung. *(Bezug: FA5, FA6, FA9)*

- Implementiert in: app/architecture.md (Zeile 27)

Kontext: 5. **BLE-Hook (useBLE)********: Kapselt die Bluetooth-Gerätekommunikation und den Reconnect.
(Erfüllt: FA3, FA5, NF2)

- Anforderung FA6: Echtzeit-Visualisierung Das System muss die empfangenen Sensordaten und Bewegungen in Echtzeit visualisieren. *(Bezug: UC-2)*

- Implementiert in: app/components/ProgressRing.tsx (Zeile 2)

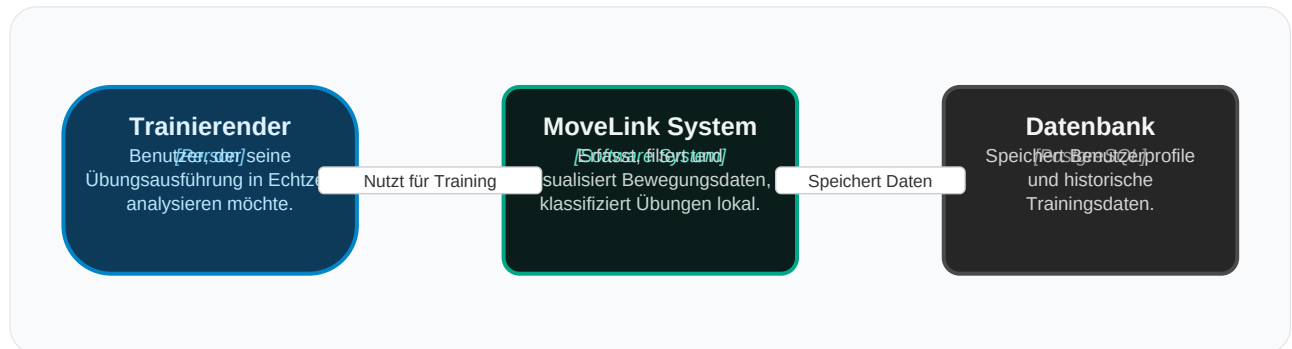
Kontext: * @implements FA6, FA9

4. System-Architektur (C4 Modell)

In diesem Abschnitt wird die Systemarchitektur auf den verschiedenen C4-Ebenen (System-Kontext, Container und Komponenten) visualisiert.

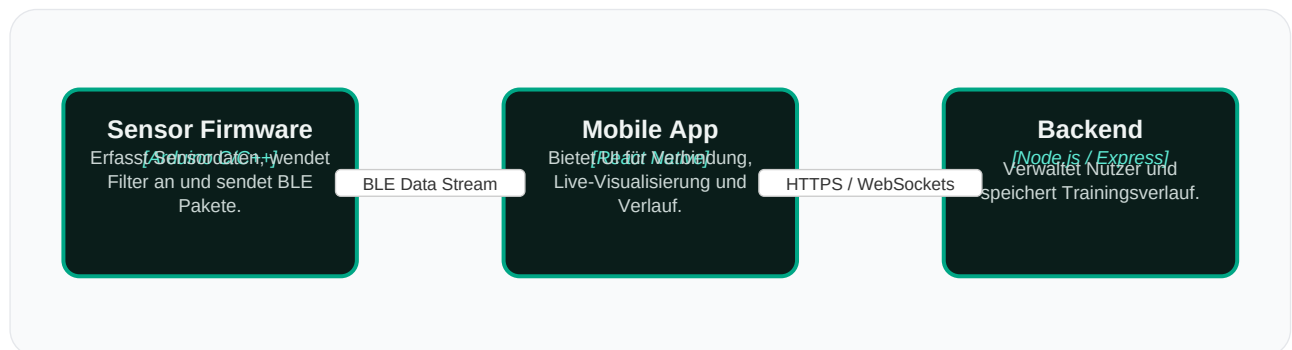
4.1 System-Kontext-Diagramm

Das System-Kontext-Diagramm zeigt die Position von MoveLink in Bezug auf Benutzer und das externe Datenbanksystem.



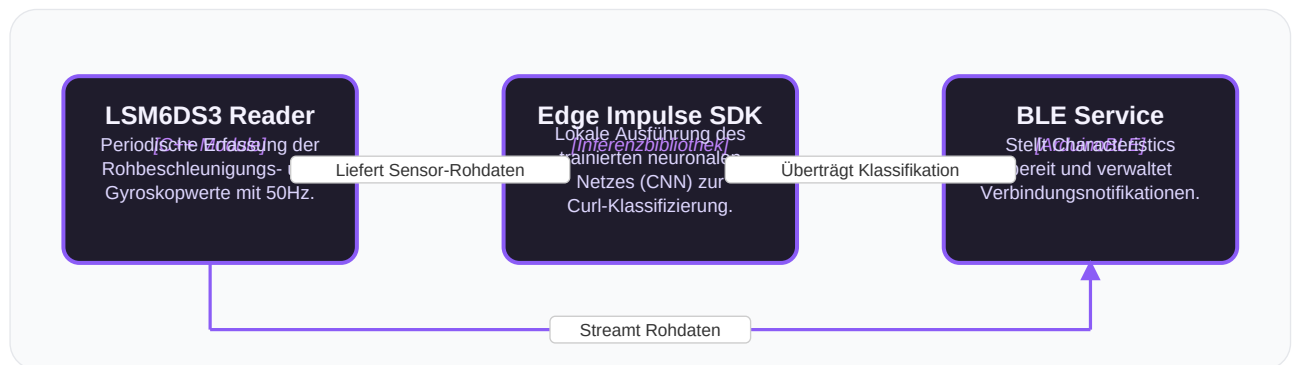
4.2 Container-Diagramm

Das Container-Diagramm veranschaulicht die Aufteilung des MoveLink Systems in eigenständige, ausführbare Subsysteme (Mobile App, Firmware und Backend) sowie deren Kommunikationspfade.



4.3 Komponenten-Diagramm (Sensor-Firmware)

Das Komponenten-Diagramm zeigt das Innenleben der Sensor-Firmware, die auf dem Xiao-Mikrocontroller läuft, sowie die Kopplung von Sensorik, Inferenz (Edge Impulse) und Bluetooth BLE.



4.4 Komponenten-Diagramm (Mobile App)

Dieses Diagramm zeigt die internen Komponenten des React Native Containers (Mobile App), aufgeteilt in UI-Elemente und Custom Hooks, die mit BLE und dem WebSocket-Server interagieren.

